

CineMAX

Manuale Tecnico

Progetto Laboratorio Interdisciplinare A

Progetto sviluppato in linguaggio Java per la gestione simulata di un cinema

Java

Console app

File locali

Javadoc

Focus tecnico

- struttura del progetto
- classi e responsabilità
- gestione dei dati
- algoritmi principali
- limiti tecnici e Javadoc

Autori

Lorenzo Guidi, Alberto Medizza, Davide Gallorini

Obiettivo del manuale tecnico

Documento pensato per chi deve leggere, mantenere o valutare il codice del progetto.

- Descrivere la struttura del progetto CineMAX in modo tecnico.
- Spiegare come sono divise le responsabilità tra classe principale, model, service e gestione file.
- Mostrare le strutture dati usate per tutte le classi all'interno del progetto.
- Evidenziare le scelte algoritmiche più importanti: ricerca, validazione input, prenotazione e salvataggio, in base a come meglio pensato per il progetto
- Indicare formato dei file, percorsi, limiti tecnici e gestione del progetto in modo più opportuno.

Manuale utente

spiega come usare il programma

Manuale tecnico

spiega come è costruito il programma

Nel nostro manuale tecnico il codice viene spiegato a livello progettuale: non riga per riga, ma per componenti, flussi e scelte.

Indice

1

Struttura del progetto

cartelle, package e file principali

2

Architettura applicativa

main, servizi, model e persistenza

3

Classi e responsabilità

dominio, utenti, proiezioni e prenotazioni

4

Strutture dati e algoritmi

liste, matrice sala, ricerca e salvataggio

5

Gestione dei file

formati, percorsi e aggiornamento dei dati

6

JavaDoc e limiti tecnici

documentazione generata e miglioramenti possibili



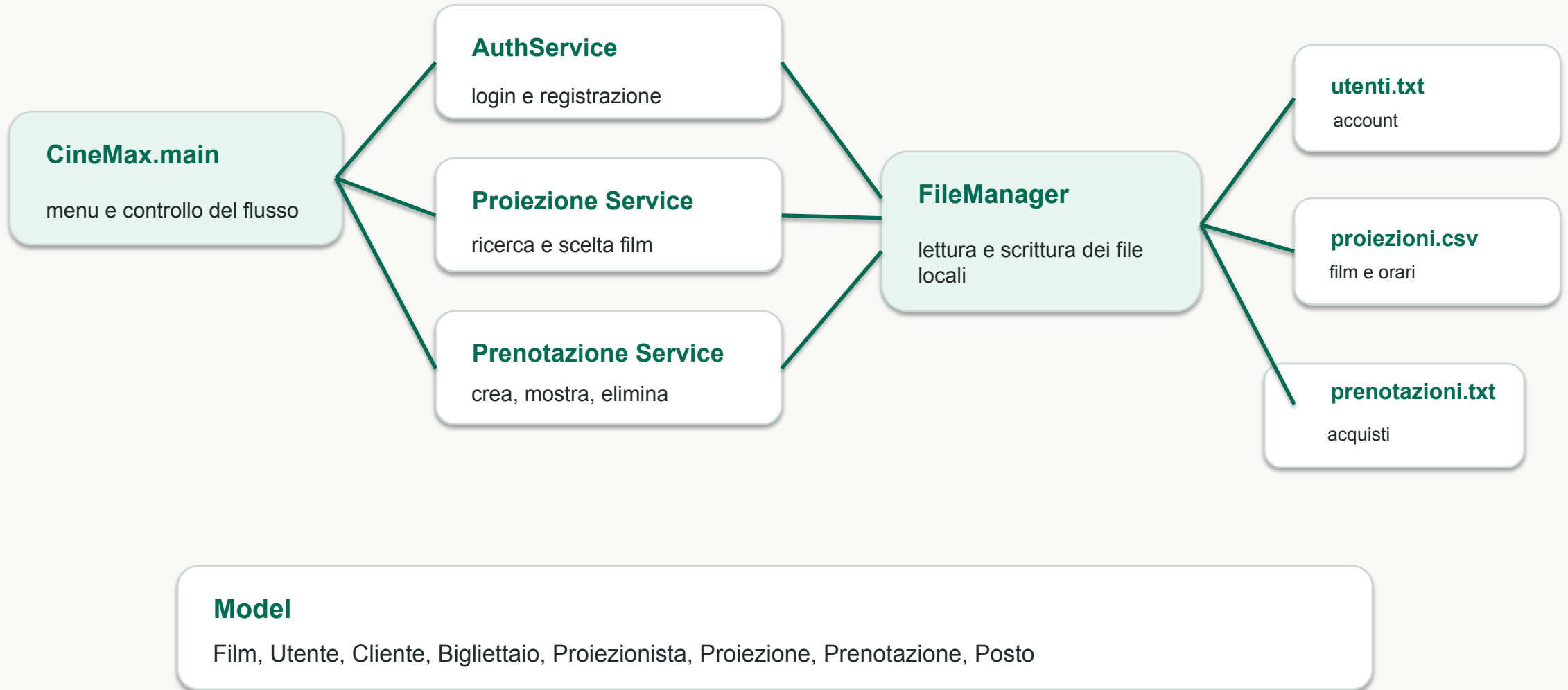
Struttura del progetto

Il codice è diviso in package semplici: interfaccia console, model, service e file dati.



- CineMax.java contiene il main e il menu testuale.
- Il package model contiene le entità del dominio: utenti, film, proiezioni, posti e prenotazioni.
- Il package service contiene la logica applicativa: autenticazione, ricerca proiezioni, prenotazioni e lettura/scrittura dei file.
- La cartella data sostituisce un database: contiene i dati letti e aggiornati dal programma.
- La cartella docs contiene la documentazione Javadoc generata dal codice commentato.

Architettura applicativa



Entry point e menu principale

La classe CineMax è il punto di partenza dell'applicazione e coordina i servizi.

CineMax.java - gestione dei servizi

```
Scanner scanner = new Scanner(System.in);
AuthService authService = new AuthService();
ProiezioneService proiezioneService = new ProiezioneService();
PrenotazioneService prenotazioneService =
    new PrenotazioneService(proiezioneService);

Utente utenteLoggato = null;

switch (scelta) {
    case 1: utenteLoggato = authService.login(scanner); break;
    case 3: proiezioneService.cercaProiezioni(scanner); break;
    case 5: prenotazioneService.creaPrenotazione(
        scanner, utenteLoggato, proiezione);
}
```

- Il main crea gli oggetti di servizio una sola volta all'avvio.
- Lo stato dell'utente loggato viene mantenuto nella variabile utenteLoggato.
- Il menu usa uno switch per richiamare il servizio corretto in base alla scelta dell'utente.
- Le funzioni di prenotazione controllano che l'utente abbia effettuato il login prima di proseguire.
- La lettura dei numeri è protetta da un metodo che accetta solo valori dentro un intervallo valido.

Package model: classi del dominio

Le classi model descrivono gli oggetti principali usati dal programma.

Classe	Responsabilità	Dati principali
Film	descrive un film	titolo, genere, regista, anno, durata, età minima
Utente	base comune degli utenti	nome, cognome, username, password, ruolo
Cliente	utente che prenota	eredita da Utente con ruolo cliente
Bigliettaio / Proiezionista	ruoli previsti dal sistema	eredita da Utente con ruolo specifico
Proiezione	film in una certa data e ora	Film, data, ora, costo, posti disponibili
Prenotazione	acquisto fatto da un cliente	codice, username, proiezione, posti, totale
Posto	posizione nella sala	riga e colonna

Strutture dati principali

Il progetto usa strutture semplici, leggibili e adatte a un'applicazione didattica.

ArrayList<Proiezione>

contiene tutte le proiezioni caricate da proiezioni.csv

ArrayList<Prenotazione>

contiene le prenotazioni caricate e aggiornate durante l'esecuzione

ArrayList<Posto>

salva i posti scelti dentro una singola prenotazione

boolean[10][20]

rappresenta la sala: false = posto libero, true = posto occupato. La dimensione fissa crea 200 posti.

Proiezione.java

```
private boolean[][] posti;  
  
// 10 righe x 20 colonne = 200 posti  
posti = new boolean[10][20];  
  
if (!posti[riga][colonna]) {  
    posti[riga][colonna] = true;  
    postiDisponibili--;  
}
```

La scelta della matrice rende la verifica di un singolo posto molto immediata: basta controllare la cella corrispondente a riga e colonna.

Package service: logica applicativa

I service fanno da ponte tra menu, model e file.

AuthService

legge username e password,
verifica le credenziali, registra
nuovi clienti usando
FileManager

ProiezioneService

carica le proiezioni, stampa
l'elenco e cerca film per titolo o
parte del titolo

PrenotazioneService

crea prenotazioni, calcola il
totale, mostra ed elimina
prenotazioni

FileManager

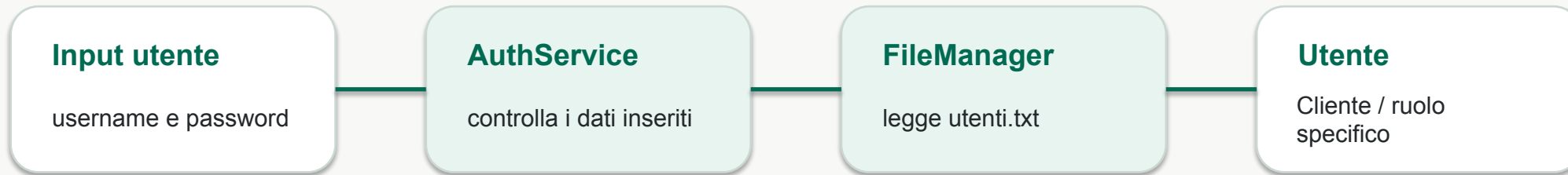
legge e scrive utenti,
proiezioni e
prenotazioni

```
costanti e percorsi principali  
private static final String UTENTI_FILE = "data/utenti.txt";  
private static final String PRENOTAZIONI_FILE =  
    "data/prenotazioni.txt";  
  
listaProiezioni = fileManager.caricaProiezioni(  
    "data/proiezioni.csv"  
);
```

- La logica non è tutta nel main: il menu richiama i servizi.
- FileManager centralizza l'accesso ai file, evitando duplicazioni nella lettura e scrittura.

Flusso tecnico: login e registrazione

AuthService gestisce l'interazione con l'utente, FileManager si occupa dei dati su file.



```
login
if (fileManager.verificaCredenziali(UTENTI_FILE, username,
password)) {
    Utente utente = fileManager.caricaUtente(UTENTI_FILE,
username);
    return utente;
}
```

```
registrazione
boolean salvato = fileManager.salvaUtente(
    UTENTI_FILE, nome, cognome, username,
    password, dataNascita, luogoNascita,
    cliente.getRuolo()
);
```

Nel progetto le password vengono salvate in chiaro

Flusso tecnico: ricerca e selezione proiezioni

La ricerca viene fatta scorrendo la lista delle proiezioni già caricate in memoria.

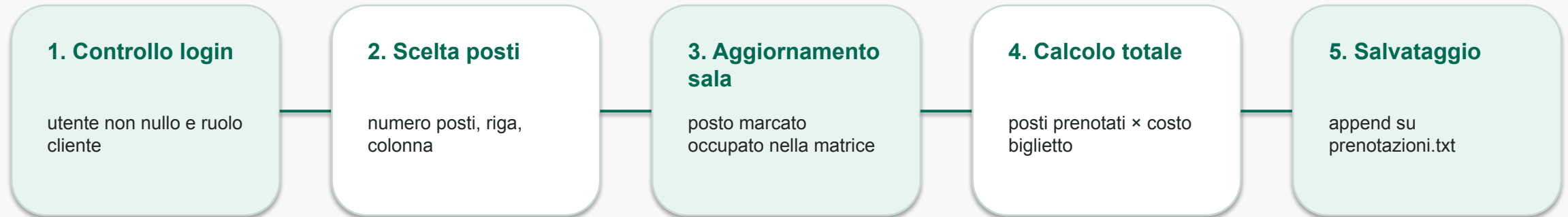
- All'avvio ProiezioneService chiede a FileManager di caricare data/proiezioni.csv.
- Ogni riga del CSV viene convertita in un oggetto Film e poi in un oggetto Proiezione.
- La ricerca confronta il testo inserito con il titolo del film, usando contains e ignorando maiuscole/minuscole.
- La selezione della proiezione avviene tramite indice numerico mostrato nel terminale.

ProiezioneService.java - ricerca film

```
for (Proiezione p : listaProiezioni) {  
    if (p.getFilm()  
        .getTitolo()  
        .toLowerCase()  
        .contains(ricerca.toLowerCase())) {  
        System.out.println(p);  
        trovato = true;  
    }  
}
```

Flusso tecnico: prenotazione dei posti

La prenotazione aggiorna sia gli oggetti in memoria sia il file delle prenotazioni.



scelta e controllo posto

```
boolean successo = proiezione.prenotaPosto(riga,
colonna);

if (successo) {
    Posto posto = new Posto(riga, colonna);
    prenotazione.aggiungiPosto(posto);
} else {
    System.out.println("Posto già occupato!");
    i--;
}
```

calcolo e salvataggio

```
double totale = prenotazione
    .getPostiPrenotati()
    .size() * proiezione.getCosto();

prenotazione.setCostoTotale(totale);
prenotazioni.add(prenotazione);
fileManager.salvaPrenotazione(PRENOTAZIONI_FILE,
prenotazione);
```

Formato dei file e gestione dei dati

I file locali sono il sistema di persistenza del progetto.

File	Separatore	Contenuto	Uso nel programma
data/utenti.txt	;	nome, cognome, username, password, data, luogo, ruolo	login, registrazione e profilo
data/proiezioni.csv	,	data/ora, titolo, genere, regista, anno, durata, età minima, prezzo	caricamento catalogo e ricerca
data/prenotazioni.txt	; + per posti	codice, username, film, data, ora, posti, totale, data acquisto	storico prenotazioni e cancellazione

```
utenti.txt
Mario;Rossi;rossi;1234;;;cliente
Anna;Bianchi;bianchi;1234;;;proiezionista
```

```
prenotazioni.txt
codice;username;titolo;data;ora;posti;totale;dataAcquisto
8b4f...;mrossi;Good Will
Hunting;29/12/2027;16:00;A5|A6;17.00;2026-07-02T15:30:11
```

Scelta tecnica: è una soluzione leggera e facile da leggere, ma meno robusta di un database relazionale.

FileManager: lettura, parsing e salvataggio

Questa classe concentra la parte più tecnica della persistenza su file.

- Usa `BufferedReader` e `FileReader` per leggere riga per riga.
- Per il CSV delle proiezioni usa `parseCsvLine`, così evita di spezzare i titoli che contengono virgole dentro le virgolette.
- Per utenti e prenotazioni usa `split(";")` perché il formato è più semplice.
- Quando salva, controlla che la cartella data esista e aggiunge una riga nuova se il file non termina già con invio.
- Per eliminare una prenotazione, riscrive tutto il file prenotazioni.txt a partire dalla lista aggiornata.

parsing CSV

```
private String[] parseCsvLine(String line) {  
    ArrayList<String> fields = new ArrayList<>();  
    StringBuilder current = new StringBuilder();  
    boolean inQuotes = false;  
    ...  
}
```

scrittura in append

```
try (FileWriter fw = new FileWriter(file, true);  
    BufferedWriter bw = new BufferedWriter(fw);  
    PrintWriter pw = new PrintWriter(bw)) {  
    pw.println(...);  
}
```

Scelte algoritmiche principali

Gli algoritmi sono volutamente semplici: leggibili, lineari e adatti al numero di dati del progetto.

Operazione	Come funziona	Costo indicativo
Ricerca film	scorre tutte le proiezioni e confronta il titolo	$O(n)$
Login	scorre utenti.txt fino a trovare username e password	$O(n)$
Scelta posto	controlla una cella della matrice boolean[10][20]	$O(1)$
Visualizza prenotazioni	scorre la lista e filtra per username	$O(n)$
Elimina prenotazione	trova la prenotazione, libera i posti e riscrive il file	$O(n)$

Perché va bene in questo progetto

I dati sono pochi e l'obiettivo è mostrare il funzionamento, non ottimizzare per migliaia di utenti.

Quando non basterebbe più

Con molti dati servirebbero database, indici e query invece di leggere file interi e liste lineari.

JavaDoc e documentazione del codice

Il codice contiene commenti Javadoc su classi, attributi e metodi.

- Le classi principali hanno una descrizione dello scopo e i tag `@author`.
- I metodi documentati indicano parametri, valore di ritorno e ruolo del metodo.
- La cartella docs contiene la documentazione generata in HTML.
- La Javadoc aiuta chi legge il progetto a capire l'API interna senza dover analizzare subito tutto il codice.

esempio Javadoc

```
/**
 * Gestisce le operazioni relative alle prenotazioni.
 * <p>
 * Permette ai clienti di creare, visualizzare ed eliminare
 * le proprie prenotazioni, aggiornando i dati memorizzati
 * nel sistema.
 *
 * @author Davide Gallorini
 * @author Guidi Lorenzo
 * @author Alberto Medizza
 */
public class PrenotazioneService { ... }
```

Comando tipico

```
javadoc -d docs src/cinemax/**/*.*.java
```

Limiti tecnici e possibili miglioramenti

Questi limiti non bloccano il prototipo, ma sono importanti per una versione più completa.

Sicurezza

password salvate in chiaro
nei file

Persistenza

file locali semplici, visto che
non abbiamo previsto
l'utilizzo di database

Interfaccia

solo console: funziona, ma è
meno comoda di una GUI

Controlli

alcune validazioni sono
essenziali e non coprono
tutti i casi limite

Ruoli

sono previsti più ruoli, ma le
funzioni principali sono
concentrate sul cliente

Scalabilità

ricerche e salvataggi lineari
sono funzionali per il
progetto in modo pulito

Sitografia / Bibliografia

Per la parte tecnica sono stati usati il materiale didattico del corso, gli appunti delle lezioni e il codice sorgente del repository CineMAX. Le risorse online sono state usate solo per verificare sintassi Java, Javadoc e gestione dei file.

- Materiale didattico universitario del corso di Programmazione / Laboratorio Interdisciplinare A.
- Appunti personali presi durante le lezioni.
- Codice sorgente del progetto CineMAX conservato su GitHub.
- Documentazione ufficiale Java per Javadoc e classi di I/O.
- Guide base sulla lettura e scrittura di file in Java.